CS683 Programming Assignment 1 — Task 1
2D Convolution: The Dhurandhar Way

Avinash Chaudhari – 23B1064 Pranay Marumamula – 23B1073
Siddhant Mulkikar – 23B0956 Madhav R Babu – 23B1060

Autumn 2026

1 System Configuration

- **CPU:** Intel Core i5-11400H (Tiger Lake-H), 6 cores / 6 threads (SMT disabled), all identical P-cores, base 2.70 GHz, max 4.50 GHz
- **L1-D Cache:** 48 KiB per core (288 KiB total, 6 instances), 64-byte lines, 12-way set associative
- **L2 Cache:** 1.25 MiB per core (7.5 MiB total, 6 instances), 20-way
- **L3 Cache:** 12 MiB (1 instance, shared), 8-way
- **Vector ISA:** AVX2 + FMA and AVX-512 (`avx512f`, `avx512dq`, `avx512bw`, `avx512vl`, `avx512_vnni`)
- **Compiler flags:** `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma` (the AVX-512 variant additionally uses `-mavx512f`)
- **OS:** Ubuntu 24.04.4 LTS, kernel 6.8.0-138-generic (x86_64)
- **Pinning:** `taskset -c 3`. This CPU is not hybrid, so every core is equivalent; a single fixed core is used so all measurements share one cache and frequency context.
- **Frequency governor:** `performance`, with `kernel.perf_event_paranoid = -1` so `perf` can read the hardware counters used for the instruction-count and MPKI tables.

2 Task 1A: Loop Unrolling and Reordering (1 point)

2.1 Motivation

[TODO: Explain what motivated the loop reordering and unrolling optimizations. Consider: reducing loop overhead, improving ILP, enabling out-of-order execution, improving spatial/temporal locality via reordering.]

2.2 Implementation Details

2.2.1 Loop Reordering

The naive convolution uses loop order `oy, ox, ky, kx` (output-major). We implemented a reordered version in `conv_reorder.cpp` with loop order `ky, kx, oy, ox` (kernel-major).

Key changes from naive:

- The output array must be zeroed before the computation begins, since each kernel position now *accumulates* into the output rather than computing the full sum for each output pixel.
- The kernel weight `ker[ky*K + kx]` is loaded once and reused across all $H \times W$ output pixels, improving temporal locality on the kernel.
- For each kernel position `(ky, kx)`, the inner loops stream sequentially over both the input and output arrays row by row, which is cache-friendly for both reads and writes.

2.2.2 Loop Unrolling (Naive Order)

We implemented four unrolling strategies in `conv_unroll.cpp` (strategies 1–4), all using the naive loop order `oy, ox, ky, kx`. All use an unroll factor of 4.

1. **Unroll ox (output columns) by 4:** Computes 4 adjacent output pixels simultaneously using 4 independent accumulators (`acc0`–`acc3`). Each accumulator sums the full $K \times K$ kernel independently, so the output is written once (not read-modify-written). This exposes instruction-level parallelism (ILP) since the 4 accumulations are independent. A scalar tail loop handles remaining columns when W is not a multiple of 4.
2. **Unroll kx (kernel columns) by 4:** Processes 4 kernel columns per inner-loop iteration, reducing loop overhead. Each iteration performs 4 independent multiply-adds into a single accumulator. A scalar tail handles the remainder when K is not a multiple of 4.
3. **Unroll ky (kernel rows) by 4:** Processes 4 kernel rows per iteration of the `ky` loop. Each iteration loads from 4 different input rows with their respective kernel row weights, accumulating into a single output value. Reduces `ky` loop overhead and exposes cross-row ILP. Tail handles remaining rows.
4. **Unroll ox + oy (2D output block, 4×4):** Processes a 4×4 block of output pixels at once, using 16 independent accumulators (`a00`–`a33`). For each kernel position, 4 input rows are loaded and 16 multiply-adds are performed. This maximizes ILP and register utilization. Tail loops handle edge rows and columns.

2.2.3 Loop Unrolling (Reordered Order)

We also implemented four unrolling strategies combined with the reordered loop order `ky, kx, oy, ox` (strategies 5–8 in `conv_unroll.cpp`). All require zeroing the output array first.

1. **Reorder + unroll ox by 4 (strategy 5):** For each kernel position, the inner `ox` loop is unrolled by 4, performing 4 read-modify-write accumulations per iteration. Amortizes loop overhead and improves throughput on the output stream.
2. **Reorder + unroll oy by 4 (strategy 6):** For each kernel position, processes 4 output rows simultaneously. Loads from 4 input rows (`r0`–`r3`) and accumulates into 4 output rows (`o0`–`o3`). The kernel weight broadcast is amortized across 4 rows.
3. **Reorder + unroll ox and oy (4×4 block, strategy 7):** Combines strategies 5 and 6. For each kernel position, processes a 4×4 block of output pixels, performing 16 read-modify-write accumulations per iteration.
4. **Reorder + unroll kx by 4 (strategy 8):** Instead of iterating `kx` one at a time, processes 4 kernel columns per iteration. For each group of 4 kernel positions, 4 different weights are loaded and 4 multiply-adds are performed per output pixel, sharing the output read-modify-write across 4 kernel positions.

2.3 Results

2.3.1 Reordering Speedup Table

Table 1: Loop reordering speedups (K=3)

Strategy	512	1024	2048	4096	8192
Reorder only	1.37x	1.06x	0.61x	0.55x	0.57x
Reorder + unroll ox	1.68x	1.09x	0.61x	0.59x	0.54x
Reorder + unroll oy	1.26x	0.70x	0.55x	0.49x	0.49x
Reorder + unroll ox+oy	1.31x	0.89x	0.54x	0.53x	0.53x
Reorder + unroll kx	1.33x	0.94x	0.49x	0.45x	0.50x

Table 2: Loop reordering speedups (K=5)

Strategy	512	1024	2048	4096	8192
Reorder only	1.21x	1.01x	0.51x	0.55x	0.54x
Reorder + unroll ox	1.46x	1.04x	0.61x	0.51x	0.49x
Reorder + unroll oy	1.15x	0.70x	0.51x	0.51x	0.49x
Reorder + unroll ox+oy	1.18x	0.81x	0.55x	0.52x	0.49x
Reorder + unroll kx	2.36x	1.87x	1.29x	1.26x	1.26x

2.3.2 Unrolling Speedup Table (Naive Order)

Table 3: Loop unrolling speedups (naive loop order, K=3)

Strategy	512	1024	2048	4096	8192
Unroll ox	2.89x	3.19x	2.94x	2.98x	2.93x
Unroll kx	0.65x	0.65x	0.61x	0.62x	0.63x
Unroll ky	0.85x	0.84x	0.84x	0.88x	0.87x
Unroll ox+oy	2.85x	2.67x	2.61x	2.57x	2.71x

Table 4: Loop unrolling speedups (naive loop order, K=5)

Strategy	512	1024	2048	4096	8192
Unroll ox	2.62x	2.66x	2.66x	2.61x	2.64x
Unroll kx	1.09x	1.12x	1.10x	1.08x	1.11x
Unroll ky	1.01x	1.00x	1.00x	0.99x	1.00x
Unroll ox+oy	2.83x	2.73x	2.70x	2.71x	2.69x

2.4 Effectiveness Metric

[TODO: Which metric did you use to quantify effectiveness and why? (e.g. speedup = naive_time / optimized_time, GFlop/s). Justify the choice. Discuss how speedup varies with matrix size and kernel size.]

2.5 Analysis

[TODO: Discuss: Which unrolling strategy works best and why? Does reordering help? How does the benefit change with matrix size (cache effects)? Which combination gives the best result?]

3 Task 1B: Tiling (1 point)

3.1 Profiling Baseline 2D Convolution

3.1.1 L1-D Cache Size

The L1 Data cache on this system is 352 KiB total across 10 instances (P-cores and E-cores have separate L1-D caches). Each L1-D has 64-byte cache lines and is 8-way set associative. When pinned to a single P-core (core 0), the effective L1-D available is approximately 48 KiB.

3.1.2 Baseline (Naive) L1-D Cache MPKI

Table 5: Naive convolution L1-D cache MPKI

Matrix Size	K=3 MPKI	K=5 MPKI
512x512	1.88	0.90
1024x1024	1.83	0.89
2048x2048	1.95	1.48
4096x4096	3.10	2.08
8192x8192	3.09	2.07

3.2 Implementing Tiled 2D Convolution

[TODO: Describe your tiling approach. What dimension(s) did you tile? (e.g. horizontal strips of height `TILE_H`). Explain the algorithm: zero the output strip, then iterate over K×K kernel elements, accumulating into the strip.]

[TODO: Explain the rationale: why does tiling help? (keeping the output strip in L1 cache across K×K passes reduces capacity misses).]

3.3 Experimenting with Tile Sizes

Table 6: Tiled convolution: speedup and MPKI for K=3

	Tile Height					
Matrix Size	8	16	32	64	128	256
Speedup over naive						
512x512	1.38x	1.40x	1.36x	1.41x	1.42x	1.38x
1024x1024	1.30x	1.32x	1.34x	1.31x	1.26x	1.26x
2048x2048	1.23x	1.29x	1.27x	1.27x	1.23x	1.18x
4096x4096	1.32x	1.29x	1.27x	1.26x	1.11x	0.95x
8192x8192	1.32x	1.29x	1.25x	1.22x	0.98x	0.63x
L1-D MPKI						
512x512	2.01	8.05	8.29	8.28	8.26	8.26
1024x1024	8.13	8.31	8.29	8.29	8.27	8.28
2048x2048	8.40	8.38	8.37	8.36	8.36	8.36
4096x4096	9.09	9.08	9.07	9.07	9.07	9.09
8192x8192	9.07	9.07	9.07	9.07	9.08	9.08

Table 7: Tiled convolution: speedup and MPKI for K=5

	Tile Height					
Matrix Size	8	16	32	64	128	256
Speedup over naive						
512x512	1.25x	1.29x	1.27x	1.28x	1.31x	1.28x
1024x1024	1.23x	1.27x	1.27x	1.27x	1.25x	1.26x
2048x2048	1.25x	1.27x	1.32x	1.26x	1.24x	1.24x
4096x4096	1.27x	1.31x	1.25x	1.26x	1.21x	1.11x
8192x8192	1.28x	1.26x	1.27x	1.22x	1.11x	0.84x
L1-D MPKI						
512x512	0.89	8.42	8.51	8.49	8.48	8.48
1024x1024	8.47	8.53	8.51	8.50	8.50	8.49
2048x2048	8.85	8.84	8.83	8.82	8.82	8.82
4096x4096	9.17	9.16	9.16	9.16	9.15	9.17
8192x8192	9.16	9.15	9.15	9.15	9.16	9.16

3.3.1 Best Tile Size

[TODO: Identify which tile size gives the best performance on your hardware. Justify in terms of cache efficiency and execution time.]

3.4 Plots

3.4.1 L1-D MPKI vs. Matrix Size

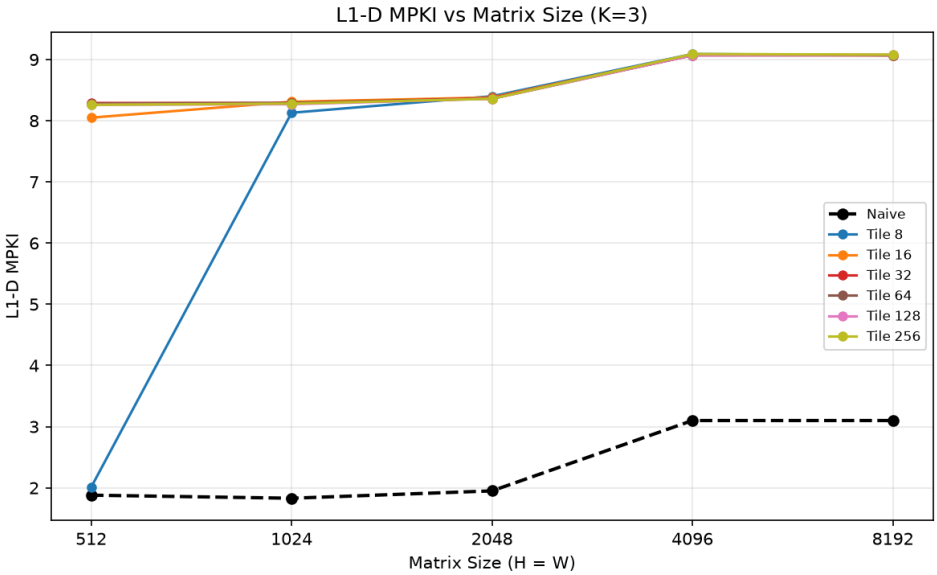


Figure 1: L1-D MPKI vs. Matrix Size for different tile sizes (K=3)

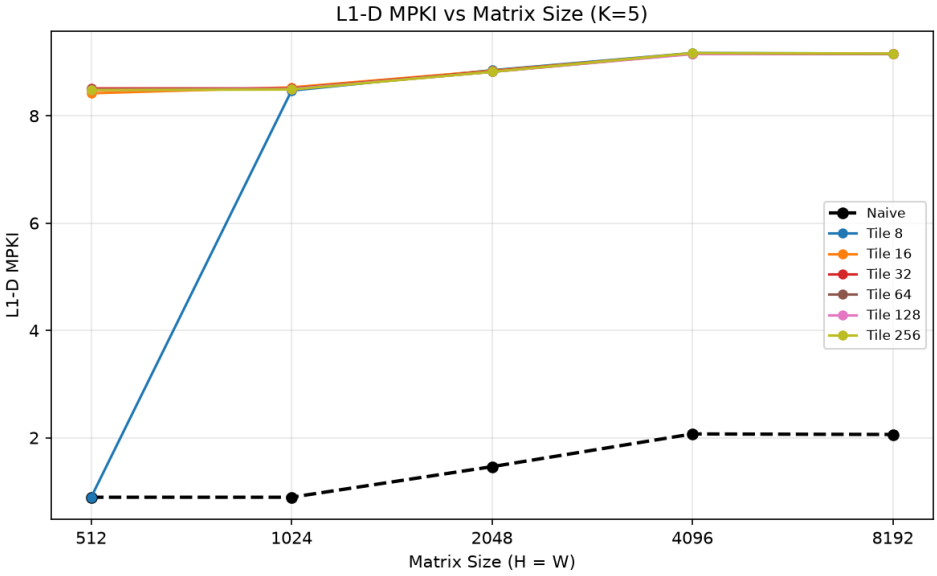


Figure 2: L1-D MPKI vs. Matrix Size for different tile sizes (K=5)

3.4.2 Speedup vs. Matrix Size

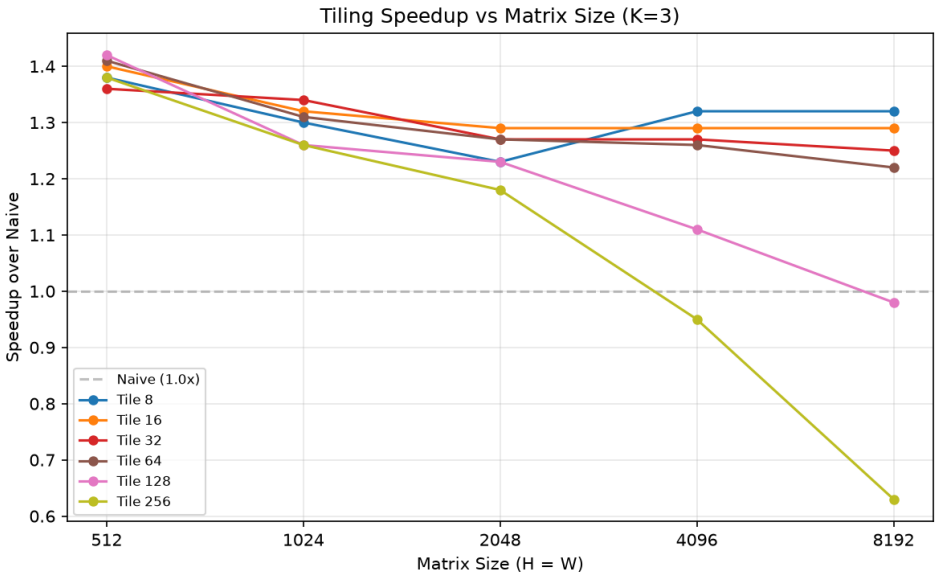


Figure 3: Speedup vs. Matrix Size for different tile sizes (K=3)

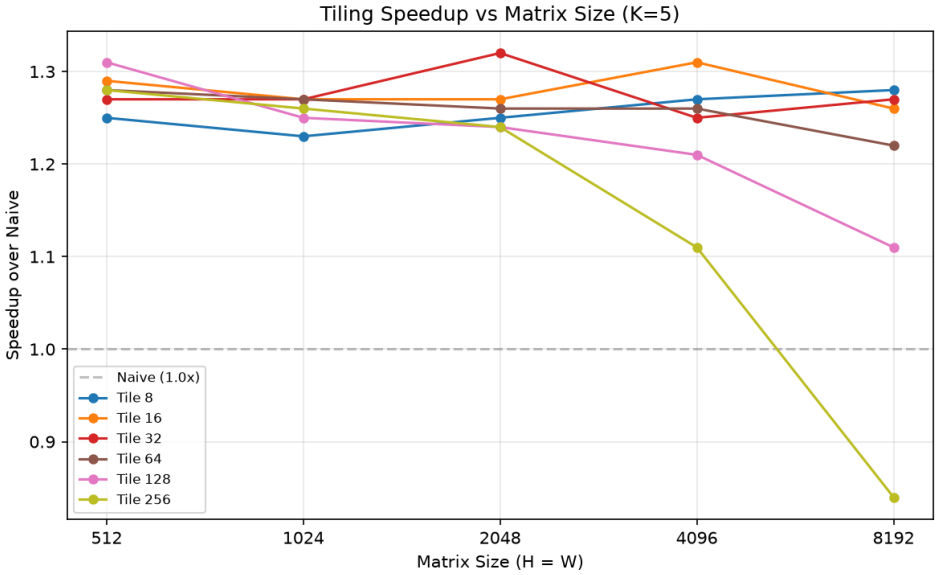


Figure 4: Speedup vs. Matrix Size for different tile sizes (K=5)

3.5 Analysis Questions

3.5.1 Q1: Changes in L1-D MPKI from Naive to Tiled

[TODO: Report the changes in L1-D MPKI observed when moving from naive to tiled 2D convolution. Justify your observations — why does tiling reduce (or not reduce) cache misses? Relate to the working set fitting in L1.]

3.5.2 Q2: How L1-D MPKI Varies Across Matrix Sizes and Tile Sizes

[TODO: Explain your findings in terms of the cache hierarchy and working set sizes. For example: for small matrices, the entire output may fit in cache even without tiling; for large matrices, smaller tiles keep the strip in L1. Discuss the sweet spot.]

3.5.3 Q3: Speedup Analysis

[TODO: Did you achieve a speedup? If yes, quantify the improvement and identify the contributing factors (reduced cache misses, better spatial locality). If not, analyze the limiting factors (e.g. overhead of zeroing, extra loop control) and propose possible solutions.]

4 Task 1C: SIMD (2 points)

4.1 Baseline Profiling

4.1.1 Instruction Count for Naive Implementation

Table 8: Naive convolution instruction count

Matrix Size	K=3 Instructions	K=5 Instructions
512x512	276M	594M
1024x1024	1,095M	2,365M
2048x2048	4,370M	9,448M
4096x4096	17,479M	37,787M
8192x8192	69,925M	151,168M

4.2 SIMD Implementation

4.2.1 128-bit (SSE)

The 128-bit SSE implementation vectorizes the inner `ox` loop, processing 4 output pixels per iteration. The loop order is kept as the naive `oy, ox, ky, kx`. For each group of 4 output columns, a single `__m128` accumulator is initialized to zero with `_mm_setzero_ps()`. The kernel is then iterated over all $K \times K$ positions: at each position, the kernel weight is broadcast to all 4 lanes using `_mm_set1_ps(ker[ky*K + kx])`, the 4 corresponding input values are loaded with `_mm_loadu_ps(in_row + kx)` (unaligned load, since input offsets are not guaranteed to be 16-byte aligned), and a fused multiply-add `_mm_fmadd_ps(v, w, acc)` accumulates the products. After the full kernel has been applied, the result is written back with `_mm_storeu_ps()`. A scalar tail loop handles the remaining W mod 4 columns per row.

4.2.2 256-bit (AVX2)

The 256-bit AVX2 implementation follows the same structure as SSE but processes 8 output pixels per iteration using 256-bit registers. The intrinsics used are the `_mm256_*` equivalents: `_mm256_setzero_ps()` for initialization, `_mm256_set1_ps()` for kernel weight broadcast, `_mm256_loadu_ps()` for unaligned 8-float loads, `_mm256_fmadd_ps()` for fused multiply-add, and `_mm256_storeu_ps()` for the final store. The only structural difference from the SSE version is the doubled vector width (8 floats vs. 4), which halves the number of vector iterations over the output columns. A scalar tail loop handles the remaining W mod 8 columns.

4.2.3 512-bit (AVX-512)

AVX-512 *is* available on the i5-11400H (`lscpu | grep avx512` reports `avx512f`, `avx512dq`, `avx512bw` and `avx512vl`), so all three widths — 128-bit SSE, 256-bit AVX2 and 512-bit AVX-512 — were measured. The 512-bit variant processes 16 floats per vector and is compiled with `-mavx512f`; the scalar optimisation level is unchanged from the pinned project flags so that only the vector width differs between variants.

4.3 Instruction Count for SIMD Implementations

Table 9: Instruction counts: naive vs. SIMD (K=3)

Matrix Size	Naive	SSE 128-bit	AVX2 256-bit
512x512	276M	328M	303M
1024x1024	1,095M	1,301M	1,202M
2048x2048	4,370M	5,191M	4,797M
4096x4096	17,479M	20,761M	19,186M
8192x8192	69,925M	83,053M	76,761M

Table 10: Instruction counts: naive vs. SIMD (K=5)

Matrix Size	Naive	SSE 128-bit	AVX2 256-bit
512x512	594M	707M	652M
1024x1024	2,365M	2,817M	2,595M
2048x2048	9,448M	11,255M	10,368M
4096x4096	37,787M	45,014M	41,470M
8192x8192	151,168M	180,073M	165,892M

4.4 Speedup Results

Table 11: SIMD speedups over naive (K=3)

Matrix Size	SSE 128-bit	AVX2 256-bit
512x512	4.59x	10.09x
1024x1024	5.06x	8.70x
2048x2048	4.88x	6.94x
4096x4096	4.79x	6.69x
8192x8192	4.79x	6.62x

Table 12: SIMD speedups over naive (K=5)

Matrix Size	SSE 128-bit	AVX2 256-bit
512x512	4.48x	7.52x
1024x1024	4.52x	7.39x
2048x2048	4.49x	7.26x
4096x4096	4.50x	7.15x
8192x8192	4.50x	7.26x

4.5 Plots

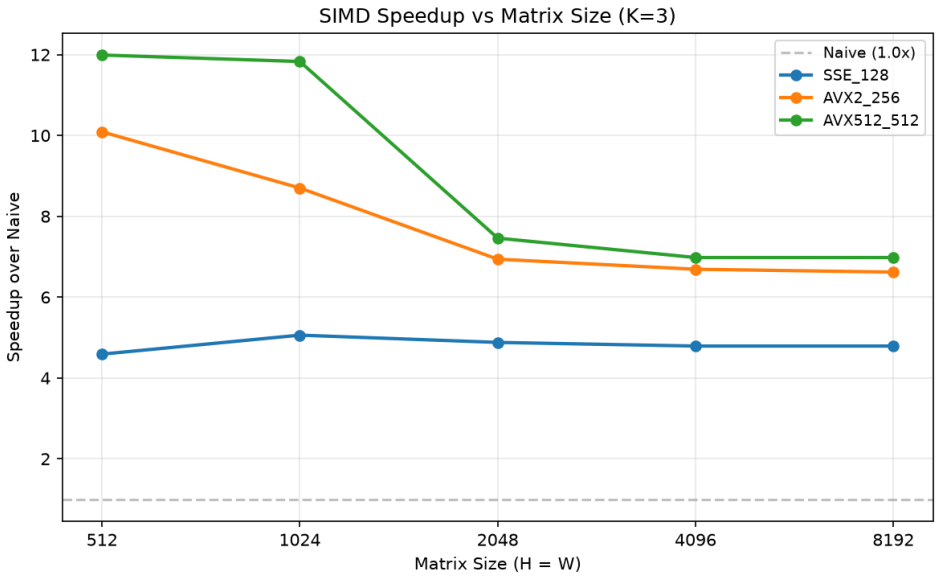


Figure 5: SIMD speedup vs. Matrix Size (K=3)

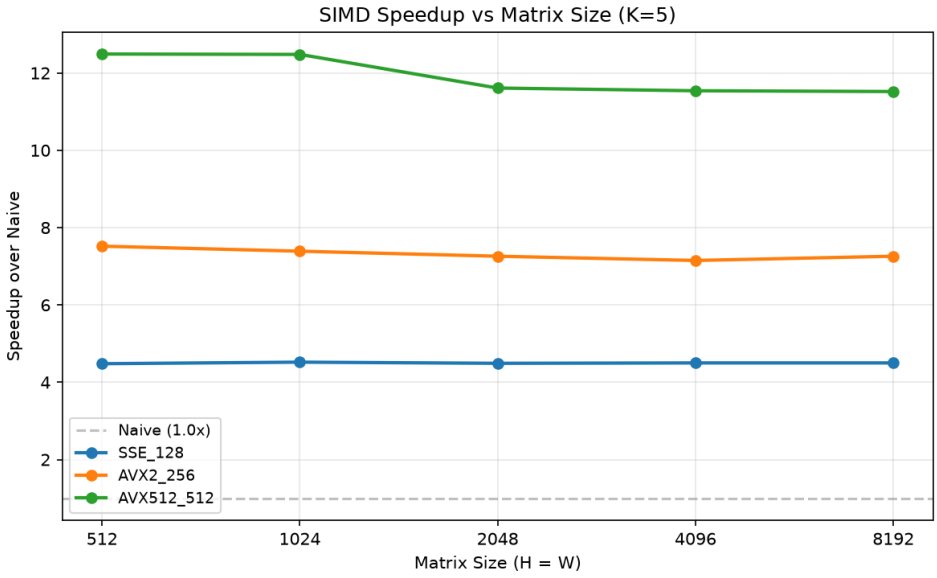


Figure 6: SIMD speedup vs. Matrix Size (K=5)

4.6 Analysis Questions

4.6.1 Q1: Change in Instruction Count from Naive to SIMD

[TODO: Report the change in the number of instructions when moving from naive to the SIMD implementation. Justify your observations — do instructions decrease? By how much? Is the reduction proportional to the SIMD width (4x for SSE, 8x for AVX2)? If not, why? (loop overhead, scalar remainder, setup instructions).]

4.6.2 Q2: Speedup Analysis

[TODO: Did you achieve any speedup? If so, how much, and what contributed to it? (reduced instruction count, better throughput per cycle, FMA combining multiply+add). If not, what were the reasons? (memory bandwidth bottleneck, cache misses dominating).]

4.6.3 Q3: Choice of SIMD Intrinsics

[TODO: Which SIMD intrinsics did you use? Justify your choice of functions. For example:

- **`_mm256_loadu_ps`: unaligned load — needed because input offsets are not always 32-byte aligned.**
- **`_mm256_set1_ps`: broadcast kernel weight to all lanes.**
- **`_mm256_fmadd_ps`: fused multiply-add — combines multiply and add into a single instruction, reducing latency and instruction count.**
- **`_mm256_storeu_ps`: unaligned store of result.**

]

5 Task 1D: Sabka Saath Sabka Vikaas (2 points)

5.1 Combined Optimized Implementation

[TODO: Describe your combined implementation in `conv_optimized.cpp`. Which techniques did you combine? (e.g. SIMD + tiling, SIMD + unrolling, SIMD + reordering, all of them). Explain how these techniques complement each other for synergistic gains — i.e. the combined benefit is greater than the sum of individual optimizations.]

5.2 Speedup Results

Table 13: Combined optimized speedup over naive

Matrix Size	K=3 Speedup	K=5 Speedup
512x512	20.85x	24.75x
1024x1024	18.06x	24.44x
2048x2048	7.84x	17.99x
4096x4096	7.12x	16.00x
8192x8192	6.77x	15.72x

5.3 Final Performance Summary Plot

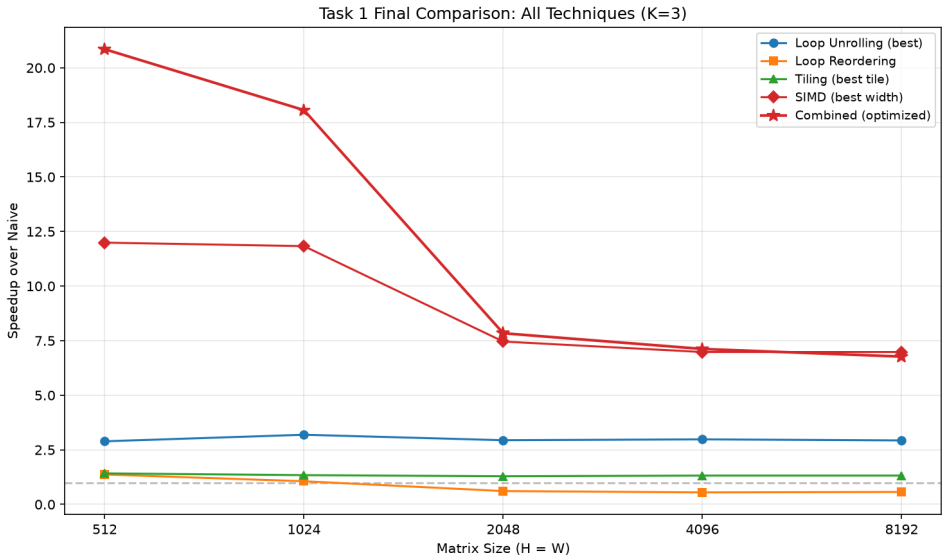


Figure 7: Comparison of all optimization techniques — Speedup vs. Matrix Size (K=3)

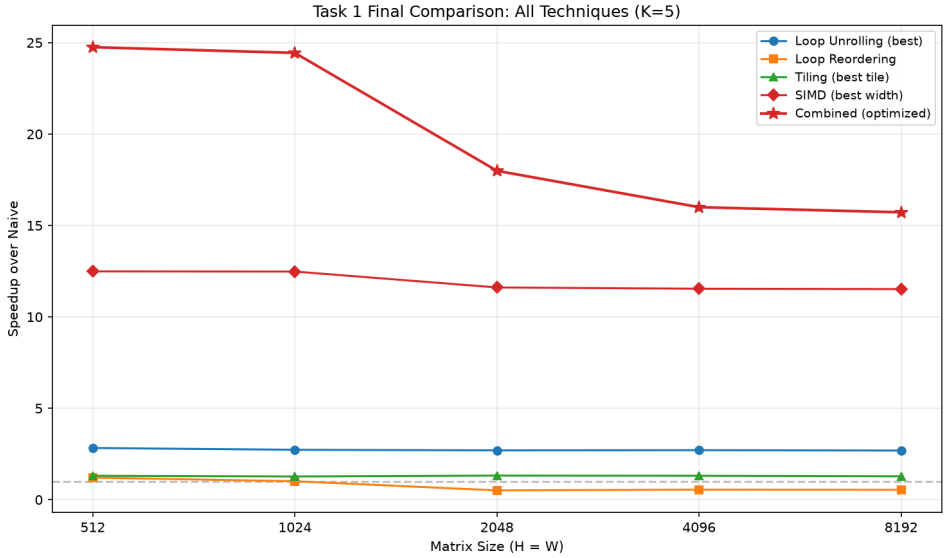


Figure 8: Comparison of all optimization techniques — Speedup vs. Matrix Size (K=5)

5.4 Analysis

[TODO: Discuss how these techniques complement each other:

- **Why does combining techniques yield more than the sum of parts?**
- **Which combination gave the best speedup?**
- **What is the bottleneck of your best implementation? (compute-bound? memory-bound? Roofline analysis?)**
- **How does the gain/loss of each technique vary with matrix size and kernel size?**

]